Arquitectura del Frontend

Sistema de Gestión de Gastos. Una Single Page Application (SPA) ágil, reactiva y modular construida con React 19, TypeScript y Vite.

Visión General y Tecnologías

Stack Tecnológico Principal

- **React 19:** Librería de UI enfocada en componentes reactivos.
- **TypeScript:** Tipado estático nativo para evitar errores en tiempo de ejecución.
- **Vite 7:** Servidor de desarrollo ultrarrápido y bundler optimizado (reemplazo de Webpack).
- **React Router 7:** Navegación dinámica (Single Page Application) sin recargar.
- **Axios:** Interfaz asíncrona para comunicación HTTP segura con la API.

Principios de Diseño

- **Modularidad Estricta:** Los dominios (auth, gastos, core) viven en carpetas independientes.
- **Atomic Design:** Metodología de escalado UI dividiendo desde *átomos* hasta *páginas*.
- **Separación de Capas:** UI (Componentes), Lógica (Context/Hooks) y Red (Servicios) nunca se mezclan.
- **Barrel Exports:** Uso sistemático de index.ts para encapsular la estructura interna.

El Ensamblador: main.tsx & App.tsx

La Estructura de Raíz

El archivo App.tsx orquesta el árbol principal del que derivan todas las funcionalidades:

- : Proveedor de historial de navegación.
- : Envuelve la aplicación inyectando el estado global del usuario (sesión).
- : Renderiza condicionalmente la Navbar y evalúa qué vista mostrar.

Protección de Rutas (Guardians)

El componente ProtectedRoute es el portero de las páginas seguras.

- Verifica isAuthenticated a través del useAuth() hook.
- Si no hay sesión válida: ejecuta .
- **Protección Inversa:** Si un usuario con sesión activa intenta visitar el /login o /register, es expulsado de vuelta a /gastos.

Modularidad por Dominio

Módulo Auth

Centraliza la identidad. Contiene el estado global de sesión (AuthContext.tsx) y los componentes visuales de entrada como LoginPage y RegisterPage.

Módulo Core

La base transversal de la app. Almacena las interfaces de TypeScript (types/), clientes HTTP genéricos (services/) y componentes de UI primarios reusables (Atoms).

Módulo Gastos

El orquestador de negocio. Integra filtros de meses (MonthFilter), interactúa con la API y gestiona el CRUD completo del flujo financiero del usuario conectado.

El Orquestador: GastosPage

Estado Reactivo de Página

GastosPage.tsx es el componente central, coordinando un estado complejo:

- expenses (array) y booleanos como isLoading o isSubmitting.
- Cálculos automáticos locales (derivados) como totalIncome, totalExpenses y balance sin recargar consultas al servidor.
- Apertura y cierre modal dictado por el puntero expenseToEdit.

Actualización Optimista (UI)

Máxima fluidez. La tabla muta localmente antes de reconsultar al Backend:

- **Crear:** setExpenses((prev) ⇒ [nuevoGasto, ...prev]); (Inserción al inicio).
- **Editar:** Mapea y reemplaza únicamente el elemento afectado coincidente por ID.
- **Eliminar:** Filtra el listado eliminando el ID borrado y actualizando los gráficos inmediatamente.

Estrategia: Atomic Design

Construcción Visual Escalable

La UI se construye combinando partículas de menor a mayor complejidad.

- **Átomos 🔵:** Piezas indivisibles y agnósticas (Ej. con props de tamaño o badges de prioridad).
- **Moléculas 🟢:** Unión de átomos para funciones específicas (Ej. = Label + Input).
- **Organismos 🟠:** Zonas complejas con estado propio (Ej. o modales complejos).
- **Templates y Pages 🟣:** Layouts que enmarcan organismos y los conectan a servicios de datos (Ej.).

Estado Global: AuthContext

Manejo con Context API

En lugar de instalar Redux, la sesión se maneja nativamente.

- El expone el user, login(), logout() y un flag de carga.

- **Persistencia Local:** En el evento "mount", lee localStorage. Si existe un token, llama silenciosamente a getProfile() para rehidratar el usuario.
- Si el token expiró, la promesa falla e invoca un logout() automático, limpiando el Storage.

El Custom Hook useAuth()

Consumo encapsulado y seguro del proveedor:

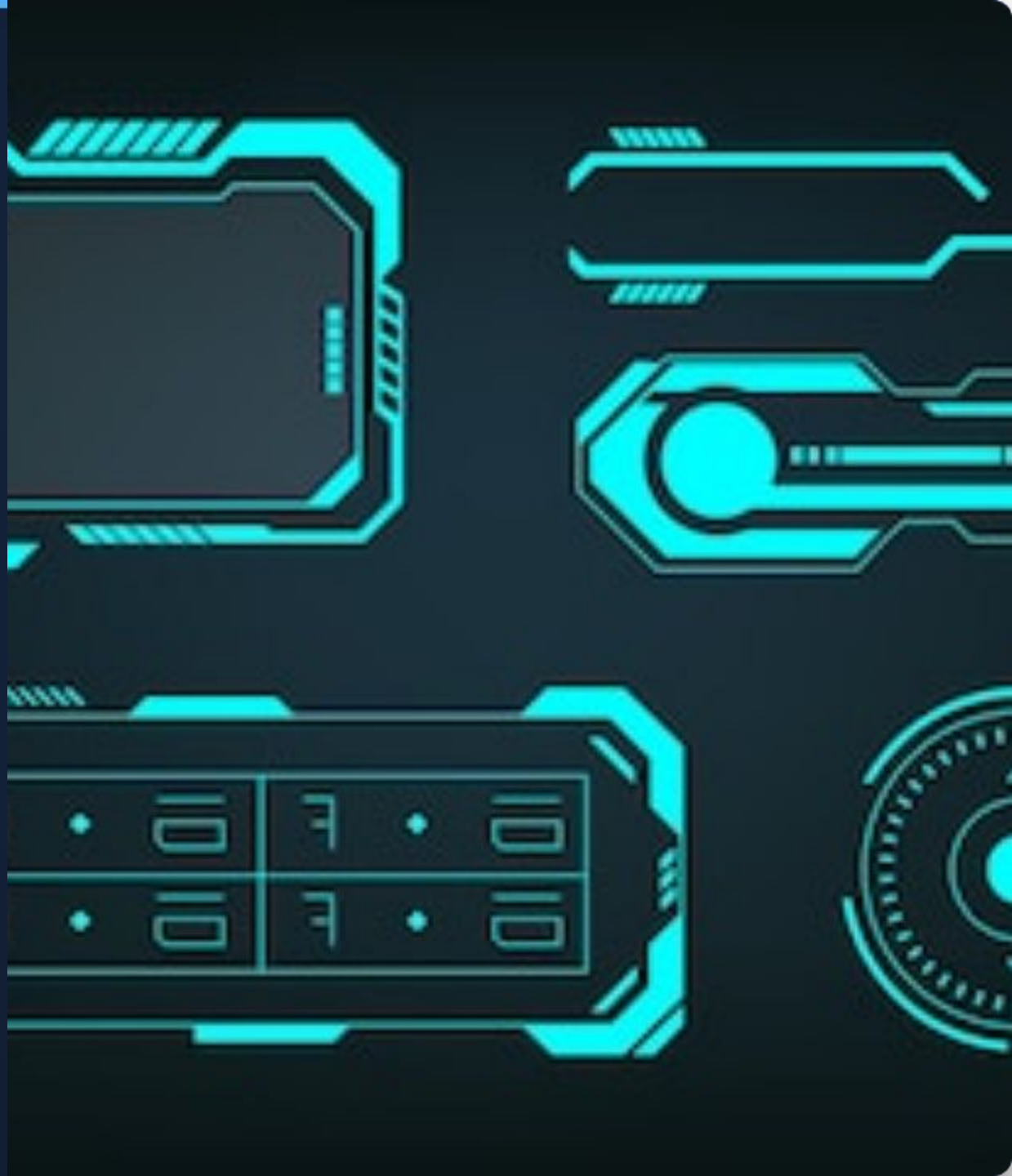
- Extrae el contexto de forma limpia: const { user } = useAuth();
- Posee un "Safety Catch" interno: si es llamado fuera del árbol del Provider, arroja una excepción clara alertando del fallo de arquitectura, previendo comportamientos de estado undefined.

Comunicación: Axios

El Interceptor de Seguridad

El frontend se comunica con el backend sin inyectar tokens de forma redundante:

- **Instancia apiClient:** Configurada apuntando dinámicamente a la variable import.meta.env.VITE_API_URL.
- **Inyección Automática:** Un interceptor captura todas las peticiones salientes, localiza el token en localStorage y lo adjunta a la cabecera Authorization: Bearer.
- **Desacoplamiento:** expenseService ejecuta llamadas (GET, POST) limpias, ignorando completamente los mecanismos de seguridad de red, los cuales son gestionados en la base del cliente.



El Espejo: Tipos de Datos

- **Duplicación Deliberada:** Los archivos de la carpeta types/ en el Frontend replican la estructura exacta definida en la Capa 1 del Backend.
- **Seguridad en Runtime:** Un IExpense frontend sabe que solo puede poseer status 'INCOME' o 'EXPENSE', permitiendo a TS detectar errores de asignación en tiempo de diseño.
- **DTOs de Transferencia:** Estructuras como CreateExpenseDTO omiten el id. Si un formulario intenta despachar el ID, el compilador lo bloquea de inmediato.
- **Desacoplamiento Seguro:** Permite que ambos proyectos se mantengan en repositorios separados compartiendo el mismo "idioma" (API Contracts).

Estabilidad y Ecosistema

Patrón Try / Catch / Finally

Todas las acciones CRUD asíncronas implementan este bloque. Capturan excepciones de red, exponen componentes al usuario, y aseguran que los spinners de carga se apaguen en la cláusula finally.

Vite Environment Vars

Segregación de entornos. Usa import.meta.env para consumir variables de .env. Vital para cambiar el origen (VITE_API_URL) de localhost a producción en el proceso de despliegue dinámico.

Scripts y Compilación

Flujo ágil con Node Package Manager: npm run dev despliega HMR (Hot Module Replacement) instantáneo, y npm run build empaqueta la SPA resolviendo el tipado y minificando JS/CSS puros.